

Practical: 7

Implementation of making a change problem using dynamic programming

Aim: Implementation of making a change problem using dynamic programming

ALGORITHM:

input :-
n = number of coins
coins[] = array of coin denominations
amount = target amount
Output:-
 • minimum number of coins
 • maximum number of coins
Steps
 1) Start
 2) Read n, the number of coins
 3) Read the denominations (into coins[])
 4) Read the target amount
 5) create two DP arrays:
 • dp_min[] for minimum coins
 • dp_max[] for maximum coins
 6) Initialize:
 • dp_min[i] = amount + 1
 • dp_max[i] = -1
 7) Set the base case:
 • dp_min[0] = 0
 • dp_max[0] = 0
 8) for Every amount i from 1 to amount:
 • for every coin coins[j]:
 • if coins[j] ≤ i:
 • calculate minimum:
 dp_min[i] = min(dp_min[i], dp_min[i - coins[j]] + 1)
 • calculate maximum:
 dp_max[i] = max(dp_max[i], dp_max[i - coins[j]] + 1)
 9) if dp_min[amount] is greater than amount, display "change can be made"
 10) otherwise display:
 • dp_min[amount] as the minimum number of coins
 • dp_max[amount] as the maximum number of coins
 11) Stop

PROGRAM

```
#include <iostream>
#include <algorithm>
using namespace std;

class CoinChange {
private:
    int coins[10];
    int n;
    int amount;

    int dpMin[100];
    int dpMax[100];

public:
    void getInput() {
        cout << "Enter number of coins: ";
        cin >> n;

        cout << "Enter coin denominations: ";
        for (int i = 0; i < n; i++) {
            cin >> coins[i];
        }

        cout << "Enter amount: ";
        cin >> amount;
    }

    void solve() {

        // Initialize DP arrays
        for (int i = 0; i <= amount; i++) {
            dpMin[i] = amount + 1;
            dpMax[i] = -1;
        }

        dpMin[0] = 0;
        dpMax[0] = 0;

        // Dynamic Programming
        for (int i = 1; i <= amount; i++) {
```

```
for (int j = 0; j < n; j++) {

    if (coins[j] <= i) {

        // Minimum number of coins
        dpMin[i] = min(dpMin[i],
                      dpMin[i - coins[j]] + 1);

        // Maximum number of coins
        if (dpMax[i - coins[j]] != -1) {
            dpMax[i] = max(dpMax[i],
                          dpMax[i - coins[j]] + 1);
        }
    }
}

// Display result
if (dpMin[amount] > amount) {
    cout << "Change cannot be made." << endl;
}
else {
    cout << "Minimum number of coins = "
         << dpMin[amount] << endl;

    cout << "Maximum number of coins = "
         << dpMax[amount] << endl;
}
};

int main() {

    CoinChange c;

    c.getInput();
    c.solve();

    return 0;
}
```

OUTPUT:

```
Output

Enter number of coins: 4
Enter coin denominations: 1
5
6
9
Enter amount: 10
Minimum number of coins = 2
Maximum number of coins = 10

=== Code Execution Successful ===
```

TIME COMPLEXITY:

There are two loops:

```
- for (int i = 1; i <= amount; i++) {
    for (int j = 0; j < A; j++) {
        // dp calculation
    }
}
```

→ outer loop runs amount times.
→ inner loop runs n times.

∴ $TCC = O(n \times \text{amount})$

for both minimum and maximum

The program calculate minimum and maximum is the same loop ; so:

$TCC = O(n \times \text{amount})$



CONCLUSION:

Thus, the Making Change Problem was successfully implemented using Dynamic Programming in C++. The program finds the minimum and maximum number of coins required to make the given amount efficiently. Dynamic Programming avoids repeated calculations by storing previously computed results, improving the overall efficiency of the solution.